

Q1 Debug Nested Exception Handling

```
import java.util.Scanner;

public class Debug1 {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);

        int arr[] = {20,40,60,80};

        int index = sc.nextInt();
        int divisor = sc.nextInt();

        try {
            int value = arr[index];
            try {
                int result = value/divisor;
                System.out.println(result);
            }
            catch(ArrayIndexOutOfBoundsException e) {
                System.out.println("Invalid Index");
            }
        }
        catch(ArithmeticException e) {
            System.out.println("Division by Zero");
        }
    }
}
```

```
}
catch(ArithmeticException e) {
    System.out.println("Division by Zero");
}
}
```

Debugging Task

Identify why some exceptions are never caught.

Modify the exception handling so all runtime exceptions are handled correctly.

TEST CASE 1

Input 2 5

Expected Output: 12

TEST CASE 2

Input 5 2

Expected Output : Invalid Index

TEST CASE 3

Input 1 0

Expected Output : Division by Zero

```
1 import java.util.Scanner;
2 class Main
3 {
4     public static void main(String[] args)
5     {
6         int[] arr = {20, 40, 60, 80};
7         Scanner sc = new Scanner(System.in);
8         if (sc.hasNextInt())
9         {
10             int index = sc.nextInt();
11             if (sc.hasNextInt())
12             {
13                 int divisor = sc.nextInt();
14                 try
15                 {
16                     int value = arr[index];
17                     try
18                     {
19                         int result = value / divisor;
20                         System.out.println(result);
21                     }
22                     catch (ArithmeticException e)
23                     {
24                         System.out.println("Division by Zero");
25                     }
26                 }
27             }
28         }
29     }
30 }
```

```

25         }
26     }
27     catch (ArrayIndexOutOfBoundsException e)
28     {
29         System.out.println("Invalid Index");
30     }
31 }
32 }
33 sc.close();
34 }
35 }

```

Q2 Debug Exception Propagation

```

import java.util.Scanner;

class Calculator{

static int divide(int a,int b){
return a/b;
}

static void display(int x){
System.out.println("Answer = "+x);
}

}

public class Debug2{

public static void main(String args[]){

Scanner sc=new Scanner(System.in);

int a=sc.nextInt();
int b=sc.nextInt();

Calculator.display(Calculator.divide(a,b));

System.out.println("Execution Completed");
}

}

```

Debugging Task

Prevent abnormal termination.

Ensure "Execution Completed" is always printed.

TEST CASE 1

INPUT: 20 5

OUTPUT : Answer = 4

Execution Completed

TEST CASE 2

INPUT: 50 0

OUTPUT: Cannot Divide by Zero

Execution Completed

TEST CASE 3

INPUT: 36 6

OUTPUT: Answer = 6

Execution Completed

Test Input

```

2 5
5 2
1 0

```

Output

```

12

```

Test Cases

Test Case 1



Test Case 2



Test Case 3



10.00 / 10 marks

```

1 class Calculator {
2     static int divide(int a, int b) {
3         return a / b;
4     }
5
6     static void display(int x) {
7         System.out.println("Answer = " + x);
8     }
9
10    public static void main(String[] args) {
11        java.util.Scanner sc = new java.util.Scanner(System.in);
12
13        if (sc.hasNextInt()) {
14            int a = sc.nextInt();
15            int b = sc.nextInt();
16
17            try {
18                Calculator.display(Calculator.divide(a, b));
19            }
20            catch (ArithmeticException e) {
21                System.out.println("Cannot Divide by Zero");
22            }
23            finally {
24                System.out.println("Execution Completed");
25            }
26        }
27        sc.close();
28    }
29 }

```

Q3 Debug Multiple Runtime Exceptions

```

import java.util.Scanner;

public class Debug3 {

    public static void main(String args[]) {

        Scanner sc=new Scanner(System.in);

        int arr[]={12,24,36,48};

        String number=sc.next();

        int index=Integer.parseInt(number);

        System.out.println(arr[index]);

    }

}

```

Debugging Task

The program may generate two different runtime exceptions.
Modify the program so that each exception is handled separately.

Test Case 1

INPUT: 2

OUTPUT: 36

Test Case 2

INPUT: ABC

OUTPUT: Invalid Number

Test Case 3

INPUT: 5

OUTPUT: Invalid Array Index

Your Fixed Code

```
1 public class Debug3 {
2     public static void main(String args[]) {
3         // Uses the inline path to avoid import statement errors
4         java.util.Scanner sc = new java.util.Scanner(System.in);
5         int arr[] = {12, 24, 36, 48};
6
7         try {
8             String number = sc.next();
9             int index = Integer.parseInt(number);
10            System.out.println(arr[index]);
11        }
12        // Handles non-integer inputs like "ABC"
13        catch (NumberFormatException e) {
14            System.out.println("Invalid Number");
15        }
16        // Handles out-of-bounds indices like 5
17        catch (ArrayIndexOutOfBoundsException e) {
18            System.out.println("Invalid Array Index");
19        }
20    }
21 }
```

Test Input

stdin input (optional)

Output



Test Cases

Test Case 1	✓
Test Case 2	✓
Test Case 3	✓

10.00 / 10 marks

✓ Submitted

Q4 Debug Finally Execution

```
public class Debug4 {
    static int calculate(int a,int b){
        try{
            return a/b;
        }
        finally{
            System.out.println("Resources Released");
        }
    }

    public static void main(String args[]){
        System.out.println(calculate(20,0));
    }
}
```

Debugging Task

Prevent program termination.

Ensure the finally block executes while handling the exception correctly.

Debugging Task

Prevent program termination.

Ensure the finally block executes while handling the exception correctly.

Test Case 1

INPUT: 20 5

OUTPUT: Resources Released

4

Test Case 2

20 0

Resources Released

Division by Zero

Test Case 3

81 9

Resources Released

9

```

1 import java.io.BufferedReader;
2 import java.io.InputStreamReader;
3 class Main {
4     static int calculate(int a, int b)
5     {
6         try
7         {
8             return a / b;
9         }
10        catch (ArithmeticException e)
11        {
12            throw e;
13        }
14        finally
15        {
16            System.out.println("Resources Released");
17        }
18    }
19    public static void main(String[] args) throws Exception
20    {
21        BufferedReader br = new BufferedReader
22        (
23            new InputStreamReader(System.in)
24        );
25        String input = br.readLine();
26
27        if (input == null) return;
28        String[] values = input.trim().split("\\s+");
29        if (values.length < 2) return;
30        int a = Integer.parseInt(values[0]);
31        int b = Integer.parseInt(values[1]);
32        try
33        {
34            int result = calculate(a, b);
35            System.out.println(result);
36        }
37        catch (ArithmeticException e)
38        {
39            System.out.println("Division by Zero");
40        }
41    }

```

✓ Submitted

Test Input

```

20 5
20 0
81 9

```

Output

```

Resources Released
4

```

Test Cases

- Test Case 1 ✓
- Test Case 2 ✓
- Test Case 3 ✓

10.00 / 10 marks

Q5 Debug the NullPointerException

```
public class Debug4 {  
    public static void main(String[] args) {  
        String name = null;  
        System.out.println(name.length());  
    }  
}
```

Expected Task

Prevent the program from crashing.

Test Case 1

INPUT: Java

OUTPUT: Length = 4

Test Case 2

INPUT: null

OUTPUT: String is Null

Test Case 3

INPUT: Programming

OUTPUT: Length = 11

Your Fixed Code

```
1 import java.util.Scanner;  
2 public class Debug4  
3 {  
4     public static void main(String[] args)  
5     {  
6         Scanner sc = new Scanner(System.in);  
7         if (!sc.hasNextLine())  
8         {  
9             System.out.println("String is Null");  
10            return;  
11        }  
12        String name = sc.nextLine();  
13        if (name == null || name.equals("null"))  
14        {  
15            System.out.println("String is Null");  
16        }  
17        else  
18        {  
19            System.out.println("Length = " + name.length());  
20        }  
21        sc.close();  
22    }  
23 }
```

✓ Submitted

Test Input

```
Java  
Null  
Programming
```

Output

```
String is Null
```

Test Cases

Test Case 1	✓
Test Case 2	✓
Test Case 3	✓

10.00 / 10 marks